Packaging youR code: a beginner's guide to R package development

Jenny Rieck
PHAC/HC R Users Lunch & Learn

29 Jul 2026

Day in the life at HC > ROEB > HPCD

- Want to be able to track compliance history of specific companies or health products
- But the database poorly designed to link incidents
  - No common identifiers
  - Rely on free-text fields entered by public or inspectors
- ATIs, media, and litigation requests regarding compliance files necessitate fast response time
- Developed pipeline of R functions to easily search database and standardize the search results reports for SMEs to review

Before developing a package

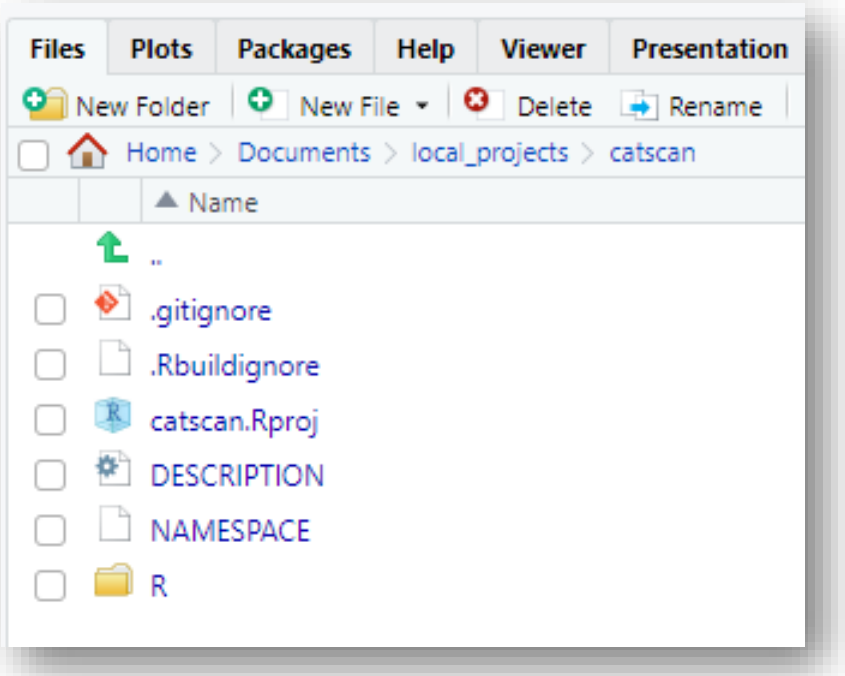
- I had put in the effort to create and refine utility functions, BUT:
  - Code was spread across scripts, projects
  - Code wasn't 100% generalizable
    - Multiple versions of the "same" function slightly tweaked
  - Code was poorly documented, lacked robust testing
- Difficulty onboarding new team members to my code
  - How to use the functions? What were the requirements? Do I have the latest version?
- Other solutions (pushing to a git repo, saving to SharePoint) did not solve all the issues

Advantages of packages

- Really a "package" deal
  - Creates one source of truth
  - As a developer, easier to maintain, manage dependencies, track versions
  - As a user, easier to integrate into a normal R workflow
- "Forces" better code practice
  - Improved reproducibility/generalizability of functions
  - Documentation, documentation, documentation!
- Strongly encourages even better practices
  - Formal unit testing
  - Writing vignettes to serve as tutorials

Getting Started

```
> install.packages("devtools")
> install.packages("roxygen2")
> devtools::create_package("mypackage")
```



Alternatively, in RStudio:

- *File > New project > New Directory > R Package*

For this session, we'll be exploring the `catscan` package

Bare minimum files

- `DESCRIPTION`
  - Meta-data about your package
    - Description, authorship, licensing, version number, dependencies
- `NAMESPACE`
  - "Generated by roxygen2: do not edit by hand"
- `myproject.Rproj`
  - RStudio specific project file
- `./R/`
  - Folder for your custom R functions

Function best practices

✅ Do

- Small and focused purpose
- Descriptive, unique, and logical naming conventions
- Write documentation
- Set default argument values
- Validate inputs
- `package::function()` approach
- Use `return()`

❌ Don't

- Rely on global variables (pass through parameters instead)
- Modify global settings:

```
> rm(list=ls())
> source()
> options()
> par()
```

```
> setwd()
> Sys.setenv()
> Sys.setlocale()
> set.seed()
```

https://www.r-bloggers.com/2024/01/r-function-writing-101a-journey-through-syntax-best-practices-and-more/

Examples of an okay vs. better function

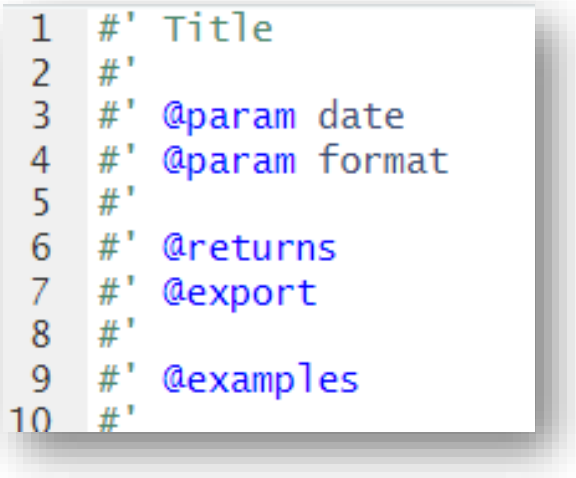
```
> library(catscan)
> fy()
> gc_fiscal_year()
```

Documenting your functions

- `roxygen2` package to help create function documentation

```
> help("mypackage::myfunction")
```

- Add roxygen comments with `#'` in your function files
  - Code > Insert Roxygen Skeleton



```
#' Title
#'
#' @param date
#' @param format
#'
#' @returns
#' @export
#'
#' @examples
#'
```

- `devtools::document()` to generate/update your .Rd help files located in `./man/`

Testing your functions

- Ad hoc testing:
  - Do not `source()` your functions, instead:

```
> devtools::load_all()
```

- Formal testing with `testthat`

```
> usethis::use_testthat()
```

  - Explicitly document a function's expected behavior
  - Automated checks

```
> devtools::test()
```

Vignettes

- Longer form, step-by-step tutorials of functions in a package and how they work together

```
> usethis::use_vignette("my-vignette")
```

- .Rmd allows you to describe your functions, insert and test code chunks, insert tables/figures

Build the package!

1. `> devtools::test()`
2. `> devtools::document()`
   - Creates `./man/*.Rd` help files for each function
3. `> devtools::build()`
   - Builds the local `package_X.X.X.tar.gz`
4. `> devtools::check()`
5. `> devtools::install(build_vignettes = TRUE)`
6. **Restart R Session**
7. `> library(mypackage)`

Sharing your package

- CRAN for the brave
  - But, strict submission and maintenance requirements
- As a repo on GitHub or GC Code

```
> devtools::install_git("user/repo", build_vignettes = TRUE) # But deprecated
> pak::pak("user/repo") # does not built vignettes by default
```

- Share the `package_X.X.X.tar.gz`

```
> install.packages("path/to/package_X.X.X.tar.gz", type = "source", build_vignettes = TRUE)
```

  - But, `*.tar.gz files` will not sync through SharePoint/OneDrive
- Alternatively, share as a `*.zip`

```
> devtools::build(binary = TRUE)
> install.packages("path/to/package_X.X.X.zip", type = "win.binary") # does not build vignettes
```

Updates and versioning

- Track package version number in `DESCRIPTION` file
  - `X.X.X = MAJOR.MINOR.PATCH`
  - `MAJOR` = incompatible/breaking changes
  - `MINOR` = added feature, functions
  - `PATCH = bug fix`
- Generally, `0.1.0` as your first working version
- Track updates in `NEWS.md`

```
> usethis::use_news_md()
```

Resources

- *R Packages*, Hadley Wickham & Jenny Bryan, https://r-pkgs.org/
- *Making your first R package*, Fong Chun Chan, https://tinyheero.github.io/jekyll/update/2015/07/26/making-your-first-R-package.html
- *Packaging Data Analytical Work Reproducibly Using R (and Friends)*, Ben Marwick, Carl Boettiger, & Lincoln Mullen, https://doi.org/10.32614/RJ-2018-058